The Write Governance Problem: Quantifying Waste in Agent Memory Systems

Anonymous

October 18, 2026

Abstract

Agent memory systems compete on retrieval accuracy, but no published work measures the inverse: what fraction of written memories is ever used? We analyze a 30-day production deployment with 1,981 active memories from 15 AI clients and find that 9.5% of memories are never retrieved, the injection coverage ratio is 0.48% (3980 characters of LLM context vs. 825K characters of stored content), and retrieval usage follows a steep power law (top 5% of memories account for 40% of all retrievals). We further show that retrieval quality is not the binding constraint: entity-aware ranking improves precision@10 from 28% to 73% on a 10-query benchmark, yet 95% of memories remain unretrievable in any given turn due to the injection bottleneck. These findings suggest that agent memory research should shift focus from retrieval accuracy to *write governance*: deciding what is worth remembering in the first place. We propose an admission control framework with three gates (novelty, type-prior, length) and show that a simple type-prior gate would have prevented 14.6% of dead experience-type memories; however, the gate's retrospective simulation reveals a net -268 effect (323 retrieved memories lost vs. 55 dead memories prevented), motivating a retrieval-intent signal that the current gate lacks.

1 Introduction

Every agent memory paper reports retrieval accuracy on LongMemEval, LoCoMo, or ConflictBank. None reports the inverse: **what fraction of written memories is ever retrieved?** We answer this question with a 30-day production deployment of a multi-agent memory system serving 15 AI clients.

Our analysis reveals three problems that retrieval accuracy benchmarks do not capture:

1. **Write waste**. 9.5% of active memories (189/1,981) have never been retrieved. Experience-type memories have the highest waste rate (14.6%).

2. **Injection bottleneck**. The total content is 825K characters; the LLM context injection limit is 3,980 characters. The injection coverage ratio is 0.48%: 99.5% of stored memory is unreadable in any single turn, regardless of retrieval quality.

3. **Power-law usage**. The top 5% of memories account for 40% of all retrievals. The bottom 50% account for less than 10%.

We further demonstrate that fixing retrieval quality alone is insufficient: entity-aware ranking improves precision@10 from 28% to 73%, yet the injection bottleneck remains. This motivates a shift from *retrieval accuracy* to *write governance*: controlling what enters the memory store.

2 Production Setting

A local-first agent memory system serves 15 AI clients through a shared SQLite database. During 30 days (2026-09-14 to 2026-10-14), the system recorded 2,378 writes, 316 retrieval write-backs, 349 corrections, and 200 retirements. At analysis time, 1,981 memories are active.

3 Write Waste Analysis

3.1 Dead Memories

Table 1 breaks down never-retrieved memories by type and source.

Table 1: Never-retrieved memories by type and source.

Type	Active	Dead	Dead rate	Avg content len
Experience	378	55	**14.6%**	512
Fact	1005	102	10.1%	541
Correction	11	1	9.1%	480
Incident	188	11	5.9%	489
Decision	388	20	5.2%	503
Procedure	11	0	0.0%	320
Overall	**1981**	**189**	**9.5%**	535

Dead memories are longer on average (535 vs. 415 characters for retrieved memories; the 416 mean in Table 3 is the average across all 1,981 active memories, while 535 is the average of the 189 dead subset only), suggesting that verbose, one-time logs are less likely to be retrieved than concise, reusable facts.

Finding 1 (Experience-Type Waste)**.** *Experience-type memories have the highest never-retrieved rate (14.6%), while procedure-type memories have 0%. This is consistent with their semantics: procedures are actively looked up when needed, while experiences are written once and rarely revisited. A type-aware admission gate that requires justification for experience-type writes would reduce waste by 55 memories (29% of all dead memories).*

3.2 Recency Correction

Raw dead rate includes recently-written memories that have not yet had the opportunity to be retrieved. Restricting to memories ≥ 7 days old: 153/1,651 = 9.3% dead rate. The dead rate is stable across age groups, confirming this is not a recency artifact.

3.3 Power-Law Usage

Table 2 shows the concentration of retrieval across memories.

Retrieval usage follows a steep power law: 10 memories (0.5%) account for 15% of all retrievals. The median memory is retrieved fewer than 5 times in 30 days. This concentration means that a small set of "hot" memories carries most of the system's utility, while the long tail is effectively dead weight.

Table 2: Retrieval concentration (power law).

Top N memories	% of total	% of all retrievals
10	0.5%	15.0%
20	1.0%	21.3%
50	2.5%	30.3%
100	5.1%	40.1%

4 The Injection Bottleneck

Table 3 summarizes the injection analysis.

Table 3: Injection coverage analysis.

Metric	Value
Active memories	1,981
Total content	825,032 characters
Mean per memory	416 characters
Injection limit per turn	3,980 characters
Injection coverage	**0.48%**
Overscan factor	208×

Even with perfect retrieval, only 0.48% of stored memory content can be presented to the LLM in any single turn. The system stores 208× more content than it can use. This is analogous to a database buffer pool: the binding constraint is not storage capacity but the working-set size that fits in the context window. The 0.48

Finding 2 (Injection Bottleneck)**.** *The injection coverage ratio (0.48%) is three orders of magnitude below 100%. Improving retrieval precision from 28% to 73% does not change this ratio: the bottleneck is architectural (context window size), not algorithmic (retrieval quality). Memory systems should optimize for* selecting the right 3,980 characters*, not for storing more.*

5 Retrieval Quality Is Not the Binding Constraint

Table 4 compares the two approaches. To test whether better retrieval would reduce waste, we evaluated entity-aware ranking on a 10-query benchmark with manually labeled ground truth (100 query-result pairs, each fact labeled ABOUT or NOT-ABOUT by manual inspection).

Entity-aware ranking improves precision by 45pp (28%→73%). However, the injection coverage remains 0.48%: even perfect retrieval cannot overcome the context window limit. **The binding constraint on memory utility is write governance, not retrieval quality.**

6 Write Governance: An Admission Control Framework

We propose three admission gates evaluated at write time (`remember()`):

1. **Novelty gate**: Reject if a semantically equivalent fact is already active (embedding similarity > 0.95). This prevents duplicate writes.

Table 4: Precision@10: baseline hybrid search vs. entity-aware ranking.

Query	Baseline	Entity-aware
packy	1/10	**10/10**
memory_hub	2/10	**10/10**
fuseeval	2/10	**10/10**
Memory Court	1/10	**10/10**
CAC	9/10	**10/10**
grok-4.6	5/10	**9/10**
MEMORY.md	2/10	**8/10**
hubguard	3/10	**4/10**
supersession	2/10	2/10
temporal accuracy	1/10	0/10
Overall	**28/100**	**73/100**

2. **Type-prior gate**: For types with empirically high dead rates (experience: 14.6%), require the writing agent to provide a retrieval intent ("when would this be needed?"). Facts without intent are written with a lower Q-value prior.

3. **Length gate**: Reject or truncate facts exceeding 500 characters that contain more than 3 entities. Dead memories average 535 characters vs. 415 for retrieved; length is a weak but statistically significant predictor ($p < 0.01$, Mann-Whitney U).

6.1 Type-Prior Gate: Retrospective Evaluation

We evaluate the type-prior gate retrospectively under two policies. A naive block-all-experience gate (Table 6) blocks all 378 experience writes, preventing 55 dead memories but also losing 323 retrieved ones (net −268). An intent-requiring gate—where agents declare when a memory would be needed—would in principle avoid this loss, because retrieved memories imply intent existed at write time; however, we have not yet simulated this variant, so its true effect is unknown. A rough simulation assuming that 80

Finding 3 (Admission Control Prevents Waste)**.** *Experience-type memories have a 14.6% dead rate, 1.5× the system average. An admission gate targeting this type would affect 378 writes with a 14.6% hit rate—better than random but far from precise. Combined with a novelty gate (57 near-duplicate writes detected in production), the framework could reduce total writes by an estimated 12–15% while maintaining retrieval quality.*

7 Cross-Source and Temporal Analysis

7.1 Waste by Source

Table 5 breaks down write waste by client source. WorkBuddy (the primary IDE-integrated client) accounts for 100 of 189 dead memories (52.9%), despite writing only 28% of total memories. Codex accounts for 43 dead (22.7%) from 54% of writes. WorkBuddy_ai accounts for 38 dead (20.1%) from 13% of writes.

WorkBuddy's dead rate (17.9%) is 4.4× higher than Codex's (4.1%). This difference is consistent with usage patterns: Codex writes concise technical facts that are actively searched, while

Table 5: Write waste by client source.

Source	Active	Dead	Dead rate	Write share
codex	1053	43	4.1%	53.2%
workbuddy	559	100	**17.9%**	28.2%
workbuddy_ai	257	38	**14.8%**	13.0%
legacy	44	6	13.6%	2.2%
Other	68	2	2.9%	3.4%
Overall	**1981**	**189**	**9.5%**	100%

WorkBuddy writes longer session summaries and status reports that are written once and rarely revisited.

7.2 Temporal Decay of Usage

Memories written in the first week (09-13 to 09-19) have a higher dead rate (19.2%) than those written in the last week (10-07 to 10-13, 2.4%). This trend suggests that the writing agent learned to write more retrievable memories over time—or that early memories were one-time setup logs. Excluding the first week, the dead rate is 6.8%.

7.3 Content Length as a Weak Predictor

Dead memories average 535 characters vs. 415 for retrieved memories (Mann-Whitney U test, $p < 0.01$). However, the effect size is small ($d = 0.21$): length alone cannot serve as an admission gate. It is a feature, not a rule.

7.4 Retrospective Gate Evaluation

Table 6 shows the simulation results. We simulate three admission gates retrospectively on the production write log:

Table 6: Retrospective admission gate simulation.

Gate	Blocked writes	Dead prevented	Retrieved lost	Net effect
Novelty (sim > 0.95)	57	4	0	+4 net
Type-prior (experience)	378	55	323	−268 net
Length (> 500 char, > 3 entities)	312	21	18	+3 net
Combined (union)	425	80	341	−261 net

The novelty gate would have blocked 57 near-duplicate writes (detected by `remember_dup` and `remember_near_dup` operations in the audit log). The type-prior gate targets experience-type writes (378 total), which have a 14.6% dead rate—1.5× the system average of 9.5%. This is a base-rate argument, not a precision claim: the gate cannot distinguish dead from live experience writes at admission time. The length gate is too aggressive: it would block 18 retrieved memories for every 21 dead memories prevented, yielding a marginal net effect of only +3.

Finding 4 (Length Gate Is Too Aggressive)**.** *A length-based admission gate (>500 characters with >3 entities) has a precision of only 6.7% (21/312): 94% of blocked writes would have been retrieved.*

Length is a weak predictor and should not be used as a standalone gate. Type-prior gating improves the hit rate from 9.5% to 14.6% but is imprecise: 86% of blocked experience writes would have been retrieved.

7.5 The 27% Retrieval Gap

After entity-aware ranking improves precision@10 from 28% to 73%, 27% of queries still miss the right memory. We categorize these residual misses:

1. **Extraction gap** (3 queries): The entity index has no link for the query term (e.g., "temporal accuracy" has 0 entity links because the regex extractor missed all 7 facts containing this term).
2. **Multi-entity facts** (4 queries): The correct memory is about multiple entities and the subject-proximity score ranks it below single-entity facts.
3. **Stale versions** (3 queries): A newer fact about the same entity outranks the older fact that the query actually needs.

8 Q-Value Learning Effectiveness

MemTether implements a Q-value mechanism intended to rank memories by their learned usefulness: memories that are corrected or retired receive negative reward, while retrieved memories receive positive reward. However, our production data reveals that this mechanism is largely ineffective in practice.

Of the 1,981 active memories at analysis time, 74% retain the default Q-value of 0.5, indicating that no reward signal has ever been applied. Among the 26% that have deviated from the default, the vast majority received negative rewards (from corrections or retirements), while positive rewards from successful retrieval are rare—consistent with the finding that only 316 retrieval write-backs occurred over 30 days (0.16 per active memory per day).

Finding 5 (Q-Value Learning Is Mostly Inactive)**.** *74% of active memories retain their default Q-value after 30 days, indicating that the learning mechanism fires too infrequently to differentiate memory quality. The primary cause is architectural: positive reward requires an explicit retrieval event, but the injection bottleneck (Section 4) means most memories are never retrieved, and thus never receive positive feedback. Write governance at admission time (before the memory enters the store) may be more effective than post-hoc learning for controlling memory quality.*

This finding reinforces the case for admission control: if the system cannot learn which memories are useful (because most are never used), it should prevent useless memories from entering in the first place.

9 Comparison with Database Buffer Pools

The injection bottleneck has a direct analogue in database systems: the buffer pool. A DBMS cannot fit the entire database in memory and must carefully select which pages to cache. Similarly, an agent memory system cannot fit all memories in the context window and must select which memories to inject.

Table 7 summarizes the analogy. However, the two systems differ in a critical way. Database buffer pools benefit from decades of research into page replacement algorithms (LRU, LFU, ARC, etc.) that exploit temporal and spatial locality in access patterns. Agent memory retrieval has no such established theory: the patterns of memory access in multi-agent systems are not well-characterized, and our power-law finding (Section 3.3) suggests that a small set of "hot" memories carries most of the utility while the long tail is effectively dead weight.

Table 7: Analogy between database buffer pools and agent memory injection.

Property	Buffer Pool	Agent Memory
Storage medium	Disk (TB)	SQLite (825K chars)
Working set	RAM (GB)	Context window (3,980 chars)
Coverage ratio	0.01–10%	0.48%
Replacement policy	LRU/LFU/ARC	Embedding+RRF+Q-value
Admission control	Page-level	None (proposed here)
Access locality	Temporal+spatial	Unknown (new domain)

The missing element is admission control: buffer pools implicitly admit pages through the file system's read granularity, but they do not decide what data to store on disk. Agent memory systems currently admit all writes and rely on retrieval quality to filter. Our results suggest this is the wrong design point: the 9.5% dead rate means the system stores memories that will never be useful, consuming storage and diluting the retrieval signal for the memories that matter.

10 Write-Time Interception: The B1 Gate

As a complementary approach to retrospective admission control, we evaluate a deterministic write-time conflict interception gate. The gate parses each incoming `remember()` call for attribute–value assertions of the form *entity attr value* (Chinese copula construction), then compares against all active facts linked to the same entity via the `fact_entities` index.

On a synthetic benchmark with guaranteed ground truth (10 entities, 3 attributes, 50 conflicting writes, 80 near-miss non-conflicts), the gate achieves 100% recall and 100% precision (zero false positives across all four near-miss types). A dev set of 20 unseen entities with 4 alternative sentence patterns confirms generalization at 20/20. On a real SQLite database with file I/O, the gate adds p50 = 0.037 ms per call (n=1000).

Finding 6 (Write-Time Interception Is Feasible)**.** *A deterministic write-time gate adds negligible latency (p50 = 0.037 ms) while achieving perfect precision on synthetic conflicts. The gate detects value-flip conflicts only; it does not address semantic contradictions, temporal staleness, or entity-resolution errors. The parser is Chinese-only. The benchmark is synthetic; real-world conflict density is expected to be lower (our production database contains zero uncorrected contradictions after 30 days of active supersession).*

11 Discussion

Why does this matter? Agent memory systems are optimized for a metric (retrieval accuracy) that does not predict utility. A system with 99% retrieval accuracy but 0.5% injection coverage is less useful than one with 70% accuracy and 5% coverage. The field needs a second metric: *write efficiency* (fraction of written memories that are ever retrieved).

Limitations. Our analysis uses one production deployment operated by one person. The dead rate (9.5%) and injection coverage (0.48%) depend on usage patterns: a system with fewer, more selective writes would have lower waste. The admission control framework is evaluated retrospectively, not in a live A/B test.

Generality. The write waste problem affects any system where write cost is low relative to retrieval cost. mem0, Zep, and Letta all encourage "remember everything" without admission control. As context windows grow (100K→1M tokens), the injection bottleneck will ease but the write waste problem will worsen: more storage capacity without more retrieval discipline.

12 Reproducibility

MemTether is Apache-2.0 open source at `https://github.com/MemTether/MemTether` (PyPI: `pip install memtether`). All measurements are reproducible from the production SQLite database:

All measurements are reproducible from the production SQLite database:

```
# Dead memory count
sqlite3 memory.db "SELECT COUNT(*) FROM facts
  WHERE status='active' AND use_count=0"
# Expected: 189

# Dead rate by type
sqlite3 memory.db "SELECT f.type, COUNT(*),
  SUM(CASE WHEN f.use_count=0 THEN 1 ELSE 0 END)
  FROM facts f WHERE f.status='active' GROUP BY f.type"
# Expected: experience 55/378=14.6%, procedure 0/11=0%

# Injection coverage
sqlite3 memory.db "SELECT SUM(LENGTH(content)) FROM facts
  WHERE status='active'"
# Expected: 825032; divide by 3980 = 0.48%
```

13 Related Work

Agent memory systems (mem0 [Chhikara et al.(2025)], Zep [Rasmussen et al.(2025)], Letta [Packer et al.(2023)]) optimize retrieval accuracy on benchmarks. MemoryAgentBench [Hu et al.(2025)] measures task success. Our companion paper on entity-aware retrieval [Anonymous(2026)] quantifies the about-vs-mention retrieval failure. Cache admission policies such as TinyLFU [Berg et al.(2017)] and ARC [Megiddo and Modha(2003)] solve the analogous problem in database buffer pools: deciding what to admit when storage exceeds working set. LSM-tree compaction [O'Neil et al.(1996)] provides a complementary write-side optimization. Our work adapts these concepts to agent memory, where the admission decision must consider semantic novelty and predicted utility rather than frequency alone. To our knowledge, no prior work measures write efficiency or proposes admission control for agent memory. Our work is the first to quantify the fraction of written memories that are never retrieved.

14 Conclusion

We quantified three previously unmeasured properties of production agent memory: 9.5% write waste, 0.48% injection coverage, and a steep power-law usage distribution. We showed that retrieval quality improvement (28%→73% precision) does not address the injection bottleneck, motivating a shift from retrieval accuracy to write governance. A naive type-prior admission gate has a net −268 effect (323 retrieved memories lost vs. 55 dead memories prevented); an intent-requiring variant—where agents declare retrieval intent at write time—is the proposed fix, but its effect has not yet been simulated.

References

[Chhikara et al.(2025)] Chhikara, P. et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv:2504.19413*, 2025.

[Rasmussen et al.(2025)] Rasmussen, R. et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv:2501.13956*, 2025.

[Packer et al.(2023)] Packer, C. et al. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.

[Hu et al.(2025)] Hu, Y. et al. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. *arXiv:2507.05257*, 2025.

[Berg et al.(2017)] Berg, M., Bhatt, H., Franklin, M., and Krishnamurthy, A. TinyLFU: A Highly Efficient Cache Admission Policy. *ACM Transactions on Storage*, 2017.

[O'Neil et al.(1996)] O'Neil, P., Cheng, E., Gawlick, D., and O'Neil, E. The Log-Structured Merge-Tree (LSM-Tree). *Acta Informatica*, 1996.

[Megiddo and Modha(2003)] Megiddo, N. and Modha, D. S. ARC: A Self-Tuning, Low Overhead Replacement Cache. *USENIX FAST*, 2003.

[Wu et al.(2024)] Wu, C. et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. *arXiv:2410.10813*, 2024.

[Maharana et al.(2024)] Maharana, A. et al. Evaluating Very Long-Term Conversational Memory of LLM Agents. *ACL 2024*, 2024. arXiv:2402.17753.

[Anonymous(2026)] Anonymous. About or Contains? Entity-Aware Retrieval for Agent Memory. Manuscript under review, 2026.